

Elite Mobile — React Native Command Reference

Cheat sheet for the day-to-day dev loop, testing, and native rebuilds on this project (Expo SDK 54, New Architecture, jest-expo, Maestro). Updated to include the test/typecheck/E2E workflow added during the prototype-fidelity work.

EVERYDAY DEV LOOP

Plain JavaScript/TypeScript changes — no native rebuild needed, the app hot-reloads over Metro.

```
# Start Metro (JS bundler) — run once, leave running in the background
npx expo start --dev-client --clear

# Type-check the whole project
npm run typecheck
# (equivalent to: npx tsc --noEmit -p tsconfig.json)
```

TESTING (JEST-EXPO)

Unit/viewModel tests using `jest-expo` + `@testing-library/react-native`. No simulator required.

```
# Run the whole suite
npx jest

# Run one file
npx jest src/modules/tickets/ui/viewModels/useTicketDetail.viewModel.test.tsx

# Run everything under a module
npx jest src/modules/roster

# Watch mode while iterating
npx jest --watch

# Verbose output (per-test names, not just suite summary)
npx jest --verbose
```

Manual mocks: native modules that crash under Jest (e.g. `expo-glass-effect`, `react-native-mmkv`) have hand-written mocks in `__mocks__` at the repo root — Jest auto-applies these, no `jest.mock()` call needed.

RUNNING ON THE IOS SIMULATOR

The normal loop for building and manually verifying a change.

```
# List available simulators
xcrun simctl list devices

# Boot one (if not already booted)
xcrun simctl boot "iPhone 17 Pro"

# Build + install + launch on the booted simulator
npx expo run:ios

# Screenshot the simulator's current screen
xcrun simctl io booted screenshot /tmp/screen.png

# Deep-link straight to a route (bypasses onboarding for quick checks)
xcrun simctl openurl booted "elitemobile://ticket-detail?ticketId=yard-prep"
```

A first-time or post-native-change build can take several minutes (CocoaPods + Xcode build). Subsequent JS-only changes hot-reload over Metro without rebuilding.

MAESTRO E2E FLOWS

Real, on-device UI flows that drive the running app via taps/text input — closes the gap that unit tests and static code review can't (actually confirms navigation, state transitions, and rendered text on a live simulator). Flow files live in `.maestro/flows/`.

```
# One-time: put maestro on PATH (installer adds this to .zshrc/.bash_profile automatically)
export PATH="$PATH":"$HOME/.maestro/bin"

# List local simulators/emulators Maestro can see
maestro list-devices

# Run one flow file
maestro test .maestro/flows/00-onboarding-and-sign-in.yaml

# Run every flow in the directory, in sequence
maestro test .maestro/flows/

# Dump the current screen's accessibility tree — essential for debugging a selector
# that won't match (shows exactly how sibling Text nodes combine into one element,
# an actual value of a field, enabled/disabled state, etc.)
maestro hierarchy
```

Gotchas: text matching is exact/anchored by default — `tap0n: "Continue"` will NOT match a button labeled "Continue to sign in"; wrap in `.*wildcards.*` for substring matching. Sibling `<Text>` elements in the same row often collapse into one combined accessibility string (e.g. a worker row becomes `"LT, Luis T., o Not clocked in"` as a single

element). Prefer `openLink: "elitemobile://<route>"` deep links over tapping the tab bar to start a flow in a known screen state. Full notes: [.maestro/README.md](#).

Known limitation: fields with `secureTextEntry` (e.g. Attestation's employee-code input) can't be reliably driven this way — iOS intentionally hides secure-field content from the accessibility APIs Maestro automates through. That's covered by unit tests instead; see [.maestro/flows/01-attestation-code-verification.yaml](#)'s header comment.

WHEN YOU NEED A NATIVE REBUILD

Only required when adding/removing a native module (an `expo-*` package, or a local module under `modules/`). Plain `.ts` / `.tsx` edits never need this.

```
# 1. Install the package the Expo-aware way (picks a compatible version)
npx expo install <package-name>

# 2. Re-link native dependencies
cd ios && pod install && cd ..

# 3. Rebuild for the simulator
npx expo run:ios
```

ANDROID BUILD & EMULATOR

This project has no `android/` native folder yet — the first `npx expo run:android` auto-generates it (same "continuous native generation" that already produced `ios/`). `ANDROID_HOME` is already exported in this machine's `.zshrc`; the `Pixel_9a` AVD already exists.

```
# One-time per shell session: the emulator binary isn't on PATH by default
# even though ANDROID_HOME/platform-tools (adb) already is
export PATH="$PATH:$ANDROID_HOME/emulator"

# List available emulator images (AVDs)
emulator -list-avds

# Boot one in the background, then leave it running
emulator -avd Pixel_9a &

# Confirm it's up (adb should list it once boot completes, ~30-60s)
adb devices

# Build + install + launch — auto-generates android/ on first run
npx expo run:android
```

First run does a full Gradle build (several minutes). Subsequent JS-only changes hot-reload over the same Metro server as iOS — no rebuild needed. If you add/remove a native module, re-run `npx expo run:android` (Gradle re-links automatically; there's no separate "pod install" equivalent step to remember).

```
# Build a debug APK directly with Gradle (equivalent to run:android but skips auto-
install/launch)
cd android && ./gradlew assembleDebug
# APK lands at: android/app/build/outputs/apk/debug/app-debug.apk

# Install a built APK onto whatever device/emulator adb currently sees
adb install -r android/app/build/outputs/apk/debug/app-debug.apk

# Uninstall (clean slate before a fresh install)
adb uninstall com.eliteteams.mobile

# Screenshot the emulator's current screen
adb exec-out screencap -p > /tmp/screen.png

# Deep-link straight to a route (same elitemobile:// scheme as iOS)
adb shell am start -a android.intent.action.VIEW -d "elitemobile://ticket-detail?ticketId=yard-
prep"

# Stream device logs (Metro's own terminal only shows JS-side logs; this shows native too)
adb logcat *:S ReactNative:V ReactNativeJS:V
```

Release builds (signed AAB for Play Store) are not yet set up on this machine — that needs a release keystore and Gradle signing config that don't exist in this project yet. When that's needed: generate a keystore (`keytool -genkeypair -v -keystore release.keystore -alias elitemobile -keyalg RSA -keysize 2048 -validity 10000`), wire it into `android/app/build.gradle` 's `signingConfigs`, then `cd android && ./gradlew bundleRelease` (produces an `.aab` under `android/app/build/outputs/bundle/release/`). Don't invent keystore credentials — this is a placeholder for when release signing is actually configured, not a command to run as-is.

PHYSICAL DEVICE (SHARON'S IPHONE 14 PRO)

Why manual `xcodebuild` instead of `npx expo run:ios --device` ? This machine's Xcode 26.6 emits a `devicectl` JSON format that this Expo CLI version's device picker can't parse, so `expo run:ios --device <id>` fails or hangs regardless of device ID. Driving `xcodebuild` + `devicectl` directly sidesteps that broken parsing step.

```
# 1. Re-link native dependencies
cd ios && pod install

# 2. Build for the physical iPhone directly
xcodebuild -workspace EliteMobile.xcworkspace -scheme EliteMobile \
```

```

-configuration Debug -destination 'id=F78E96DC-FB8E-5D23-8C19-182BE050B2E8' \
-allowProvisioningUpdates build

# 3. Install the freshly built .app
xcrun devicectl device install app --device F78E96DC-FB8E-5D23-8C19-182BE050B2E8 \
"/Users/sharonpjoseph/Library/Developer/Xcode/DerivedData/EliteMobile-
gmqksbkdydzecgkascylweiuoil/Build/Products/Debug-iphones/EliteMobile.app"

# 4. Launch it
xcrun devicectl device process launch --device F78E96DC-FB8E-5D23-8C19-182BE050B2E8 \
--terminate-existing com.eliteteams.mobile

```

HANDY ONE-OFFS

```

# List paired devices and their current devicectl identifier
# (IDs can drift across reconnects – re-check if a command reports "no device found")
xcrun devicectl list devices

# Uninstall the app (clean slate before a fresh install)
xcrun devicectl device uninstall app --device F78E96DC-FB8E-5D23-8C19-182BE050B2E8
com.eliteteams.mobile

# If Metro gets stuck or stale (rare, usually after long idle periods)
pkill -f "expo start --dev-client"
npx expo start --dev-client --clear

# Reset the simulator entirely (wipes all app data/registrations)
xcrun simctl erase "iPhone 17 Pro"

```

QUICK REFERENCE TABLE

GOAL	COMMAND
Type-check	<code>npm run typecheck</code>
Run all tests	<code>npx jest</code>
Run one test file	<code>npx jest path/to/File.test.tsx</code>
Start Metro	<code>npx expo start --dev-client --clear</code>
Build + run on simulator	<code>npx expo run:ios</code>
Install a native package	<code>npx expo install <pkg></code>
Re-link pods after install	<code>cd ios && pod install</code>

Screenshot simulator	<code>xcrun simctl io booted screenshot out.png</code>
Deep link into a route	<code>xcrun simctl openurl booted "elitemobile://<route>"</code>
Run a Maestro E2E flow	<code>maestro test .maestro/flows/<name>.yaml</code>
Dump accessibility tree	<code>maestro hierarchy</code>
Boot the Android emulator	<code>emulator -avd Pixel_9a &</code>
Build + run on Android	<code>npx expo run:android</code>
Build a debug APK only	<code>cd android && ./gradlew assembleDebug</code>
Install an APK	<code>adb install -r <path-to-apk></code>

NOTES

Device identifier `F78E96DC-FB8E-5D23-8C19-182BE050B2E8` is SHARON's iPhone 14 Pro as currently paired via `devicectl`. If it stops matching, re-run `xcrun devicectl list devices` and swap in the new identifier everywhere above.

A fresh install loses the Expo Dev Client's remembered project connection — on first open after installing, tap the running EliteMobile server under "Development servers," or enter the Metro URL manually (e.g. `http://192.168.1.33:8081`) if prompted.

This project has no test infrastructure before `jest-expo` was added — `npm test` / `npx jest` now runs 18+ suites covering entities, usecases, and viewModels across every module.